

게임 데이터베이스 분석 포트폴리오

던전앤파이터 & 리그 오브 레전드

샘플 3A 24311024

게임 시스템을 관계형 데이터베이스 관점에서 분석한 설계 포트폴리오

1. 서론

본 포트폴리오는 게임에서 사용될 수 있는 주요 데이터를 추출하여 개체(Entity), 속성(Attribute), 기본키(PK), 외래키(FK), 관계(Relationship), 카디널리티(Cardinality)를 관계형 데이터베이스 관점에서 분석한 것이다. 분석 대상은 RPG 장르의 던전앤파이터와 MORPG/MOBA 장르의 리그 오브 레전드로 선정하였다. 두 게임은 캐릭터 성장과 아이템을 다루지만, 전자는 지속적인 캐릭터·장비·던전 데이터를 중심으로 하고 후자는 경기 단위의 참여 기록과 챔피언·스킨 데이터를 중심으로 한다.

2. 게임 선정

게임	장르적 특징	선정 이유
던전앤파이터	RPG / 액션	캐릭터, 직업, 장비, 인벤토리, 던전, 몬스터, 캐시상품 등 지속형 데이터 분석에 적합
리그 오브 레전드	MOBA	챔피언, 경기, 경기 참여 기록, 룬, 스킨, RP 구매 등 경기 중심 구조 분석에 적합

3. 게임 ① 던전애파이터

던전애파이터는 캐릭터의 직업과 레벨, 장비 및 인벤토리, 던전 플레이 등 장기간 누적되는 게임 데이터를 관리해야 하는 구조로 볼 수 있다. 또한 공식 게임 가이드에서는 세라를 현금으로 충전하는 게임상 화폐로 설명하고 있으며 세라샵에서 아바타·코인·크리쳐 등의 상품을 구매할 수 있다고 안내한다. 따라서 과금 관련 데이터도 별도의 상품·구매·잔액 구조로 모델링하였다.

3.1 주요 개체 및 속성

개체	주요 속성	PK	주요 FK
ACCOUNT	account_id, email, nickname, created_at	account_id	-
CHARACTER	character_id, name, level, job_id	character_id	account_id, job_id
JOB	job_id, job_name, weapon_type	job_id	-
ITEM	item_id, item_name, item_type, price	item_id	-
INVENTORY	inventory_id, character_id, item_id, quantity	inventory_id	character_id, item_id
EQUIPMENT	equip_id, character_id, item_id, slot	equip_id	character_id, item_id
DUNGEON	dungeon_id, dungeon_name, difficulty, min_level	dungeon_id	-
CHAR_DUNGEON	character_id, dungeon_id, play_count, clear_count	character_id+dungeon_id	character_id, dungeon_id
MONSTER	monster_id, monster_name, level, dungeon_id	monster_id	dungeon_id
CASH_PRODUCT	product_id, product_name, sera_price, start_at	product_id	-
PURCHASE	purchase_id, quantity, purchased_at	purchase_id	account_id, product_id
CASH_BALANCE	balance_id, sera_amount, updated_at	balance_id	account_id

3.2 관계

관계	카디널리티	설명
ACCOUNT - CHARACTER	1 : N	한 계정은 여러 캐릭터를 생성할 수 있으며 캐릭터는 하나의 계정에 소속
CHARACTER - JOB	N : 1	여러 캐릭터가 같은 직업을 선택할 수 있음
CHARACTER - INVENTORY	1 : N	한 캐릭터가 여러 인벤토리 항목을 보유
ITEM - INVENTORY	1 : N	하나의 아이템 종류가 여러 인벤토리 기록에서 사용될 수 있음
CHARACTER - EQUIPMENT	1 : N	캐릭터의 장비 슬롯별 장착 기록
ITEM - EQUIPMENT	1 : N	아이템 종류가 여러 캐릭터의 장비로 사용될 수 있음
CHARACTER - DUNGEON	N : M	캐릭터는 여러 던전을 플레이하고 던전도 여러 캐릭터가 플레이
DUNGEON - MONSTER	1 : N	하나의 던전에 여러 몬스터가 배치
ACCOUNT - PURCHASE	1 : N	계정은 여러 번의 상품 구매를 할 수 있음
CASH_PRODUCT - PURCHASE	1 : N	하나의 상품은 여러 계정의 구매 기록에 나타남
ACCOUNT - CASH_BALANCE	1 : 1	분석 모델에서는 계정별 세라 잔액을 하나의 현재 잔액 레코드로 관리

3.3 관계형 데이터베이스 설계

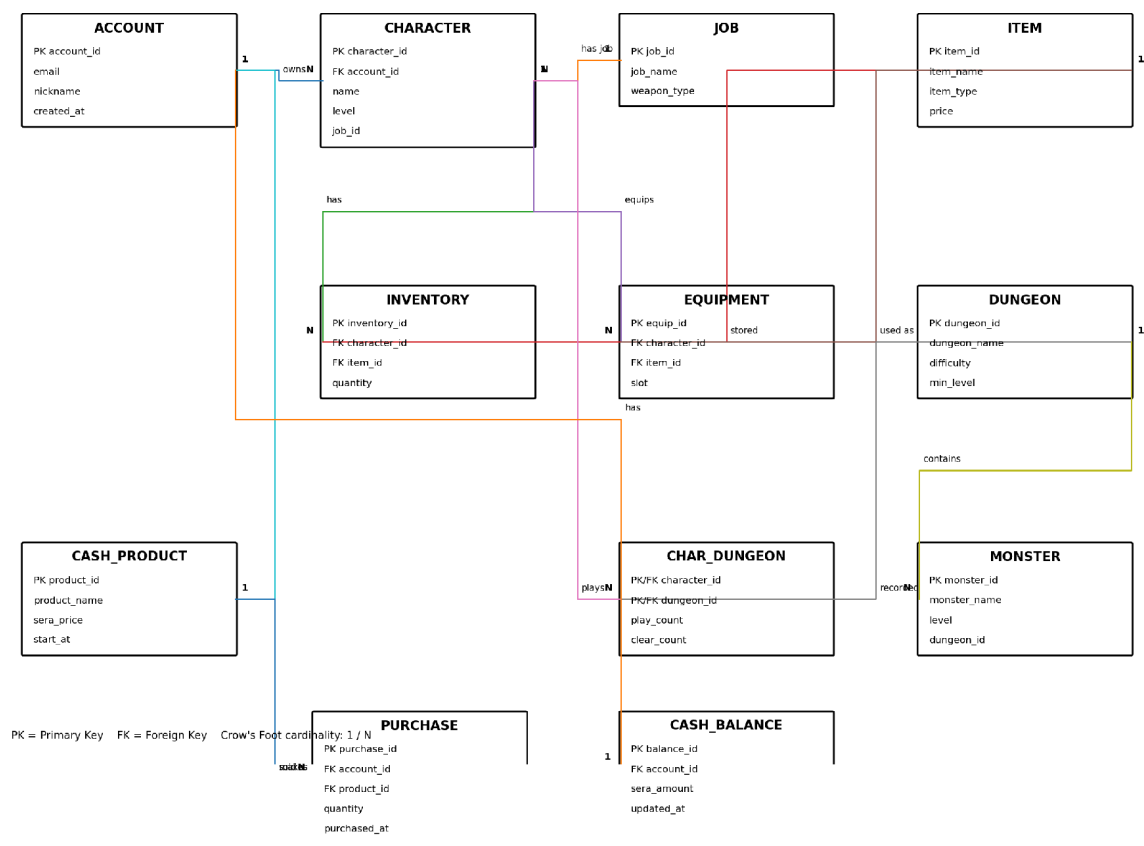
캐릭터와 던전은 다대다 관계이므로 CHAR_DUNGEON을 중간 테이블로 두었다. 이 테이블의 복합 기본키(character_id, dungeon_id)는 특정 캐릭터가 특정 던전에 대해 가진 플레이/클리어 누적 정보를 하나로 식별한다. 인벤토리 역시 캐릭터와 아이템의 연결을 기록하며 quantity를 통해 같은 아이템의 수량을 관리한다.

3.4 과금 시스템 데이터 구조

과금 부분은 ACCOUNT, CASH_BALANCE, CASH_PRODUCT, PURCHASE 네 개체를 중심으로 설계하였다. CASH_PRODUCT는 판매 상품과 가격을 저장하고 PURCHASE는 실제 구매 행위를 기록한다. CASH_BALANCE는 계정의 현재 세라 잔액을 나타낸다. 이 구조를 사용하면 '어떤 계정이 어떤 상품을 언제 몇 개 구매했는가'와 '현재 보유 세라는 얼마인가'를 분리하여 관리할 수 있다.

3.5 ERD

Dungeon Fighter Online - Analytical ERD



4. 게임 ② 리그 오브 레전드

리그 오브 레전드는 챔피언을 선택하여 한 경기 단위로 플레이하는 구조이므로, 장기간 보유되는 계정·소환사 정보와 함께 MATCH 및 MATCH_PLAYER와 같은 경기 이력 데이터가 핵심이 된다. 또한 공식 개발자 글에서는 대부분의 스킨을 상점에서 RP를 사용해 구매할 수 있다고 설명하고 있으므로 스킨·구매·RP 잔액을 별도로 모델링하였다.

출처: League of Legends 공식 개발자 블로그 – 스킨 개론

(<https://www.leagueoflegends.com/ko-kr/news/dev/skins-101/>)

4.1 주요 개체 및 속성

개체	주요 속성	PK	주요 FK
ACCOUNT	account_id, login_id, created_at, region	account_id	-
SUMMONER	summoner_id, summoner_name, level	summoner_id	account_id
CHAMPION	champion_id, champion_name, role	champion_id	-
SKIN	skin_id, skin_name, rp_price	skin_id	champion_id
MATCH	match_id, queue_type, duration, started_at, winning_team	match_id	-
MATCH_PLAYER	match_id, summoner_id, champion_id, kills, deaths, assists, win	match_id+summoner_id	match_id, summoner_id, champion_id
ITEM	item_id, item_name, cost, item_type	item_id	-
RUNE	rune_id, rune_name, path, effect	rune_id	-
PLAYER_RUNE	summoner_id, rune_id, slot	summoner_id+rune_id	summoner_id, rune_id
RP_BALANCE	balance_id, rp_amount, updated_at	balance_id	account_id
SKIN_PURCHASE	purchase_id, rp_paid, purchased_at	purchase_id	account_id, skin_id
OWNED_SKIN	summoner_id, skin_id, acquired_at	summoner_id+skin_id	summoner_id, skin_id

4.2 관계

관계	카디널리티	설명
ACCOUNT - SUMMONER	1 : N	하나의 계정에 여러 소환사 기록을 둘 수 있는 분석 모델
CHAMPION - SKIN	1 : N	하나의 챔피언에 여러 스킨이 존재
MATCH - MATCH_PLAYER	1 : N	하나의 경기에는 여러 참가자가 존재
SUMMONER - MATCH_PLAYER	1 : N	한 소환사는 여러 경기 기록에 참여
CHAMPION - MATCH_PLAYER	1 : N	하나의 챔피언이 여러 경기 기록에서 선택될 수 있음
SUMMONER - RUNE	N : M	소환사는 여러 룬을 사용하고 룬도 여러 소환사가 사용
ACCOUNT - RP_BALANCE	1 : 1	분석 모델에서 계정의 현재 RP 잔액을 관리
ACCOUNT - SKIN_PURCHASE	1 : N	계정은 여러 스킨 구매 기록을 가질 수 있음
SKIN - SKIN_PURCHASE	1 : N	동일 스킨은 여러 계정의 구매 기록에 나타남
SUMMONER - OWNED_SKIN	1 : N	소환사가 보유한 스킨을 기록
SKIN - OWNED_SKIN	1 : N	동일 스킨을 여러 소환사가 보유할 수 있음

4.3 경기 데이터 구조

MATCH는 경기 자체의 정보를 저장하고 MATCH_PLAYER는 경기와 플레이어의 연결을 담당한다.

MATCH_PLAYER에 kills, deaths, assists, win을 두면 경기별 플레이어 성과를 조회할 수 있다. 이처럼 게임 전체를 하나의 거대한 테이블로 만드는 대신 경기와 참가자 정보를 분리하면 반복되는 경기 정보의 중복을 줄일 수 있다.

4.4 과금 시스템 데이터 구조

과금 시스템은 RP_BALANCE, SKIN, SKIN_PURCHASE, OWNED_SKIN을 중심으로 구성하였다. SKIN은 상품 자체의 정보와 가격을 저장하고 SKIN_PURCHASE는 실제 결제 시점의 RP 사용량을 보존한다. OWNED_SKIN은 현재 보유 상태를 별도로 표현한다. 이 구조는 '판매 상품'과 '구매 이력' 그리고 '현재 보유 자산'을 구분한다는 점에서 데이터 관리에 적합하다.

4.5 ERD

League of Legends - Analytical ERD

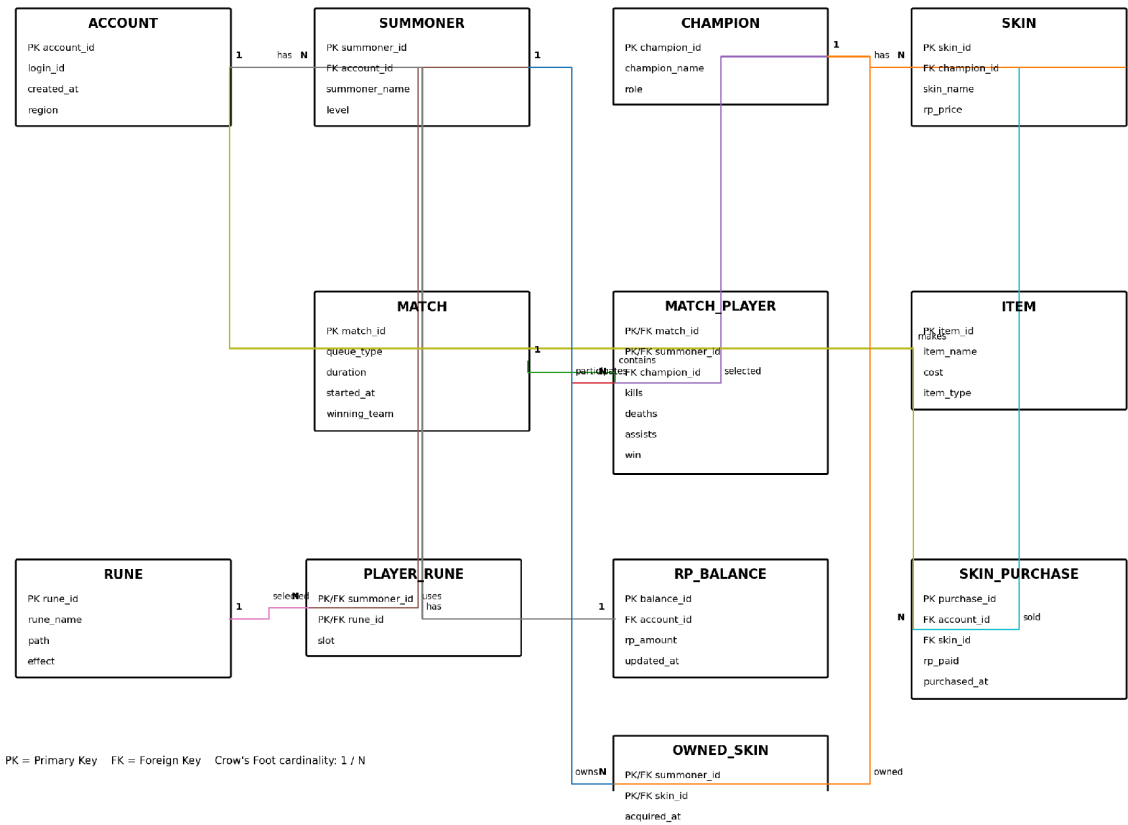


그림 2. 리그 오브 레전드 분석용 Crow's Foot ERD

5. 두 게임 데이터베이스 비교·분석

5.1 구조 비교

비교 항목	던전애파이터	리그 오브 레전드
핵심 데이터	캐릭터·직업·장비·인벤토리·던전	소환사·챔피언·경기·경기 참가
시간적 성격	장기간 누적되는 캐릭터 상태 중심	경기 단위 이력 데이터 중심
대표 N:M 관계	캐릭터-던전	소환사-룬
아이템 구조	보유 수량과 장착 상태가 중요	경기 중 선택/사용과 챔피언 조합이 중요
과금 데이터	세라·세라샵 상품·구매	RP·스킨·구매
중간 테이블	CHAR_DUNGEON	MATCH_PLAYER, PLAYER_RUNE, OWNED_SKIN
DB 설계 핵심	캐릭터 상태와 자산의 지속 관리	경기 기록의 대량 저장과 조회

5.2 공통점

두 게임 모두 계정 또는 사용자 식별 정보에서 시작하여 게임 플레이 대상과 콘텐츠 데이터를 연결하는 구조를 갖는다. 또한 과금 대상 상품과 실제 구매 이력을 분리하고, 현재 보유 자산과 구매 이력을 구분하는 것이 중요하다. 다대다 관계는 중간 테이블을 사용하여 관계를 해소하였다.

5.3 차이점

던전애파이터는 캐릭터가 계속 성장하면서 아이템과 던전 기록이 누적되는 RPG 특성 때문에 캐릭터 상태와 소유 자산을 세분화하는 것이 중요하다. 반면 리그 오브 레전드는 한 경기가 종료되면 MATCH와 MATCH_PLAYER에 결과가 남는 경기 이력 중심의 구조가 중요하다. 따라서 리그 오브 레전드는 MATCH_PLAYER와 같은 사실(fact) 성격의 테이블이 대량으로 증가할 가능성이 높고, 던전애파이터는 캐릭터·아이템·인벤토리 같은 현재 상태를 효율적으로 조회하는 것이 중요하다.

5.4 정규화 관점

게임명, 챔피언명, 아이템명과 같은 반복 정보를 경기 기록에 직접 문자열로 저장하지 않고 별도 개체로 분리하였다. 예를 들어 MATCH_PLAYER는 champion_id만 저장하고 챔피언 이름은 CHAMPION에서 가져오도록 하였다. 이는 데이터 중복을 줄이고 수정 시 데이터 불일치를 예방하는 정규화의 기본적인 목적에 해당한다.

6. ERD 작성 기준

ERD는 Crow's Foot 표기법을 사용하여 1:N 및 1:1 관계를 표현하였다. 다대다 관계는 관계형 데이터베이스에서 직접 구현하기 어렵기 때문에 연결 테이블을 추가하여 1:N + N:1 관계로 분해하였다. MySQL Workbench는 EER

Diagram에서 Crow's Foot 관계 표기를 지원하며, 관계 표기는 Model 메뉴의 Relationship Notation에서 설정할 수 있다.

참고: MySQL Workbench 공식 문서 – EER Diagram 및 Relationship Notation

(<https://dev.mysql.com/doc/workbench/en/wb-data-modeling-menus.html>)

참고: MySQL Workbench 공식 문서 – Foreign Key Relationships

(<https://dev.mysql.com/doc/workbench/en/wb-relationship-tools.html>)

7. 결론

이번 분석을 통해 같은 온라인 게임이라도 장르와 핵심 게임 플레이에 따라 데이터베이스의 중심 구조가 달라진다는 점을 확인하였다. 던전애파이터는 캐릭터와 아이템의 지속적인 성장 및 보유 상태를 관리하는 구조가 중요하고, 리그 오브 레전드는 경기와 참가자별 결과를 안정적으로 축적하고 조회하는 구조가 중요하다. 두 게임 모두 과금 상품, 구매 이력, 보유 자산을 분리하면 데이터 중복을 줄이고 거래 내역을 추적하기 쉬워진다. 따라서 게임 데이터베이스는 단순히 게임 화면에 보이는 정보를 저장하는 것이 아니라, 게임의 핵심 시스템과 이용자의 행동을 관계형 구조로 분리하고 연결하는 방향으로 설계하는 것이 중요하다.

8. 참고 자료 및 출처

- 던전애파이터 공식 게임가이드 – 세라삼

<https://df.nexon.com/guide?no=1233>

- 던전애파이터 공식 사이트 – 게임 업데이트/공지

<https://df.nexon.com/>

- League of Legends 공식 개발자 블로그 – 스킨 개론

<https://www.leagueoflegends.com/ko-kr/news/dev/skins-101/>

- MySQL Workbench 공식 문서 – EER Diagram Editor

<https://dev.mysql.com/doc/workbench/en/wb-eer-diagram-editor.html>

- MySQL Workbench 공식 문서 – Relationship Notation

<https://dev.mysql.com/doc/workbench/en/wb-data-modeling-menus.html>

- MySQL Workbench 공식 문서 – Foreign Key Relationships

<https://dev.mysql.com/doc/workbench/en/wb-relationship-tools.html>